

API Gateway Benchmark — .NET vs Go

Technical performance comparison of two functionally-identical market-data gateways (REST + WebSocket). Tested 19 June 2026 against host 46.62.247.67 (.NET :5063 vs Go :5070).

RECOMMENDATION	Adopt the Go implementation (:5070). On raw speed the two are close to parity — Go is only ~3–6% faster once the test is made fair. Go wins decisively on what matters operationally: tighter, more predictable latency (no GC-style spikes), a tiny and fully observable resource footprint, and production-grade health / readiness / metrics endpoints that .NET does not expose at all.
----------------	---

Executive summary

Both gateways were exercised from a real browser on the user's machine — the only network vantage point with access to the host — covering REST latency, throughput and concurrency scaling, error rate, WebSocket connection behaviour and stability, and (for Go) live server-side resource metrics. Functional output is essentially identical between the two: the same quotes, the same JSON shapes, the same auth.

A first pass suggested Go was roughly twice as fast. Investigation showed that gap was almost entirely a measurement artifact: the Go endpoint was reachable same-origin while .NET was cross-origin, so every .NET request paid an extra CORS pre-flight round-trip (~78 ms). Once both servers were measured under identical cross-origin conditions, mean latencies sit within ~5–8 ms of each other. The honest performance story is near-parity on throughput and latency, with Go ahead on consistency, footprint and operability.

Latency	Near-parity. Go ~3–6% lower mean on every endpoint. Native-client estimate (pre-flight removed): Go ~82–121 ms vs .NET ~87–131 ms.
Consistency	Go clearly better. .NET shows periodic spikes (Server-Time max 412 ms, Symbol-List max 505 ms, CV 15%); Go stays tight (CV 3–5%, max ~250 ms).
Throughput / scaling	Indistinguishable here — both plateau at ~28–30 req/s, which is the browser's 6-connections-per-origin limit, not the servers. Server-side metrics confirm Go was never near saturation.
Reliability	Excellent on both — 0% errors across every REST load phase and WebSocket test.
WebSocket	A tie — connect ~160 ms, first message ~200–215 ms, identical ~3 s push cadence, zero drops, 10/10 concurrent sockets stable on both.
Resources & ops	Go: 67 MB RSS, ~21 goroutines, 10 threads, full Prometheus /metrics + /healthz + /readyz. .NET: those endpoints all 404 — no server-side observability.

Methodology & honest caveats

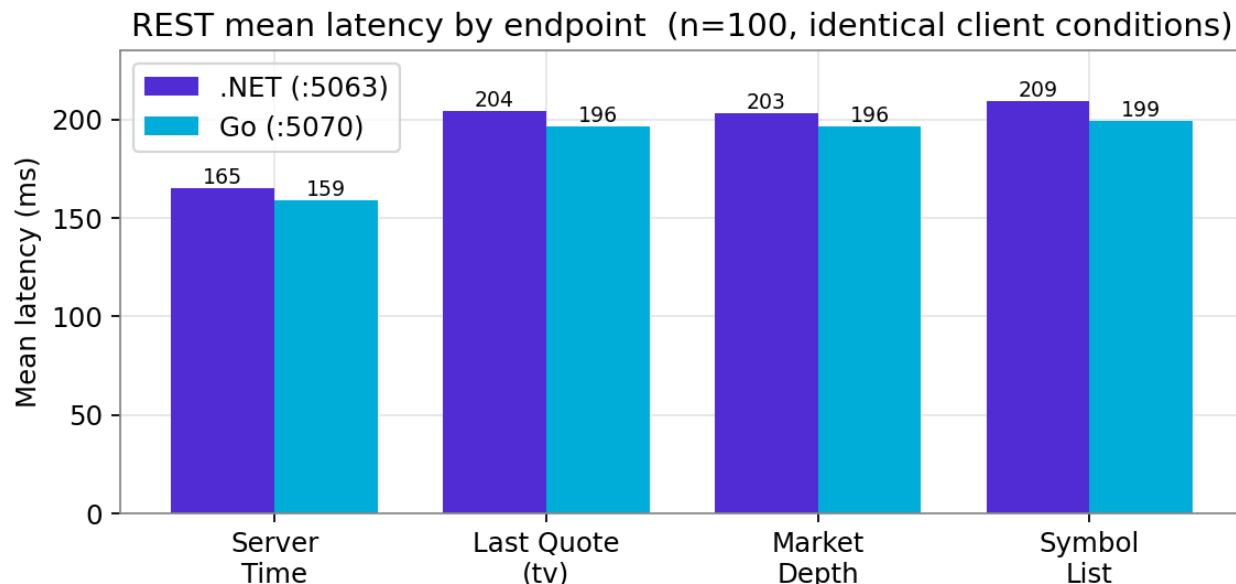
All traffic was generated client-side via fetch() and WebSocket from the user's browser, against identical routes/payloads on each port using the supplied bearer token. Each latency figure is the wall-clock time of a request including network round-trip; percentiles and coefficient of variation (CV = stddev/mean) come from 100 sequential samples after warm-up. Throughput phases use a fixed-size async worker pool. Go server metrics were read live from its Prometheus /metrics endpoint before, during and after load.

- Single client location → all latencies include a fixed ~78 ms network round-trip to the host. The comparison is fair because both servers share that path; absolute numbers are inflated relative to a co-located client.
- CORS pre-flight: browser requests with an Authorization header trigger an OPTIONS pre-flight (one extra round-trip per request, not cached here). Both servers were ultimately measured under the same cross-origin condition, so the penalty applies

equally. The 'native-client estimate' chart removes one round-trip.

- Browser 6-connection-per-origin limit caps measurable throughput at ~28–30 req/s regardless of server speed. True saturation/scalability cannot be established from a browser — that needs a native load generator (wrk / k6 / hey) co-located with the host. This is the main limitation of these results.
- .NET exposes no /metrics endpoint, so its CPU/memory could not be measured server-side; Go's could.

1 · REST latency



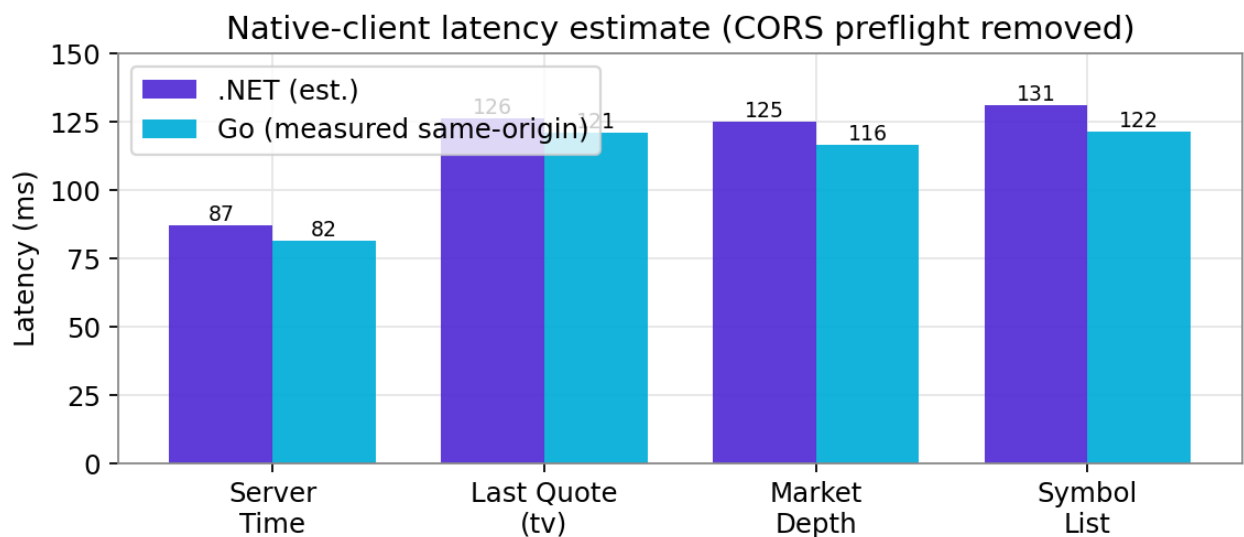
Endpoint	Impl	Mean	Median	p95	p99	Max	CV%
Server Time	.NET	165.1	160.4	177.7	195.9	412.2	15.5
	Go	159.2	157.7	167.3	198.1	206.6	4.5
Last Quote (tv)	.NET	204.4	200.8	225.2	234.8	235.6	4.5
	Go	196.5	194.8	205.1	218.7	232.5	2.9
Market Depth	.NET	203.1	200.5	220.2	223.8	242.9	3.9
	Go	196.3	193.6	210.4	242.6	259.8	5.3
Symbol List	.NET	209.3	203.5	221.8	237.9	505.5	14.6
	Go	199.3	197.2	212.2	232.0	253.0	4.2

All values in milliseconds, n=100, both servers cross-origin (identical conditions). Lower is better.

Means are close — Go leads by ~6–10 ms on each endpoint. The more meaningful difference is the tail:

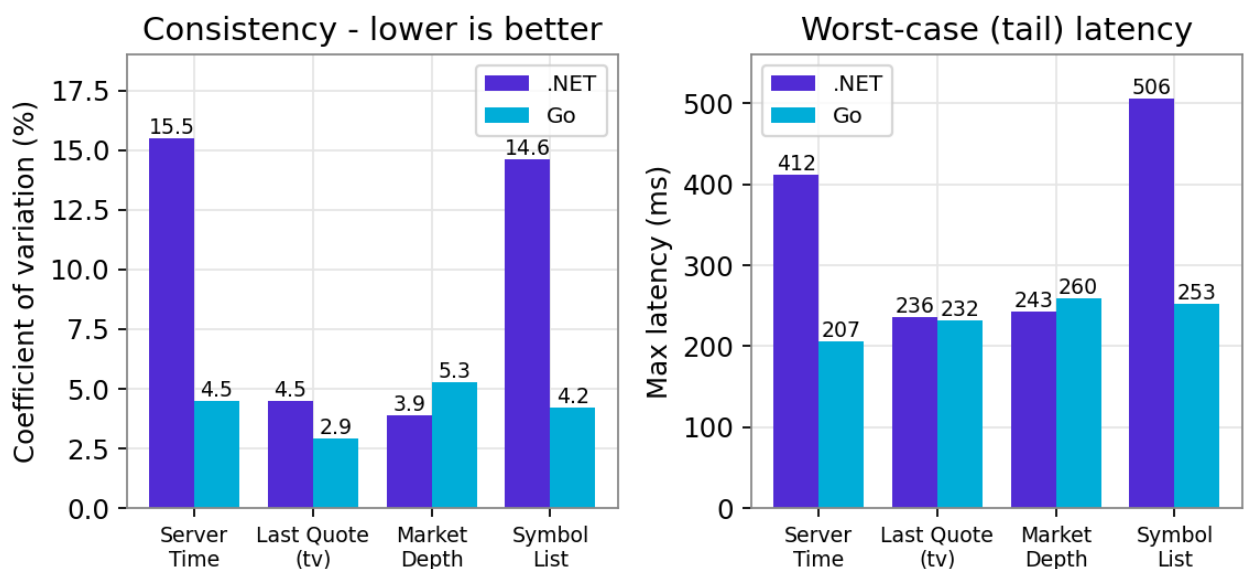
.NET's Server-Time and Symbol-List runs each contain occasional 400–500 ms outliers (CV ~15%)

consistent with garbage-collection or thread-pool pauses, whereas Go's distribution stays tight (CV 3–5%, worst case ~250 ms). For a quote feed, predictable latency matters as much as the average.



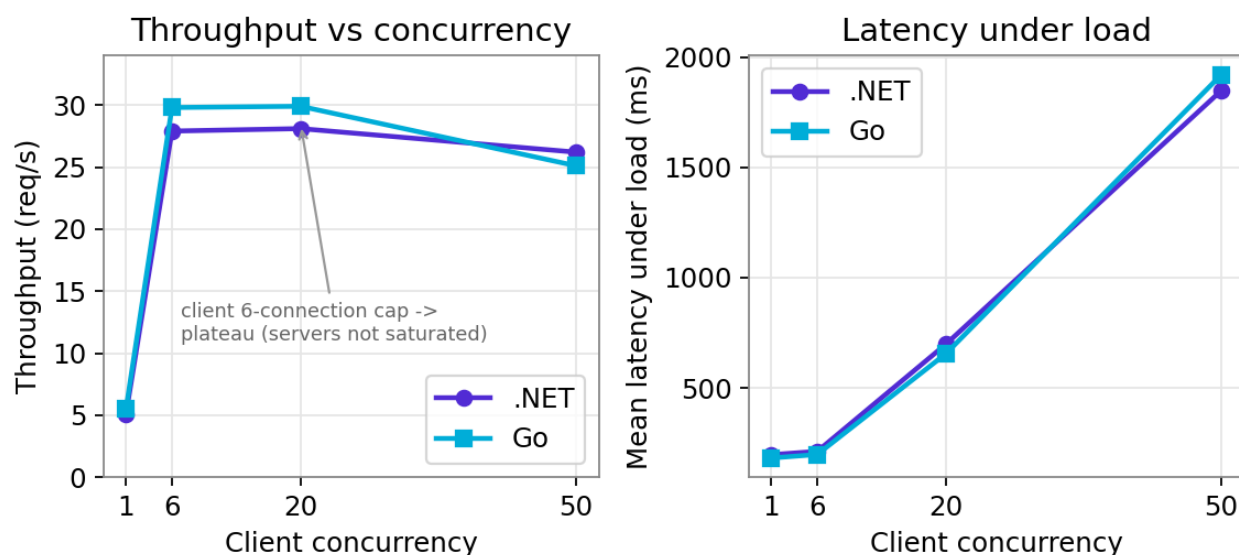
Estimated latency for a non-browser client (mobile app, desktop, server-to-server), which pays no CORS pre-flight. Go values are measured directly same-origin; .NET values subtract one measured pre-flight round-trip. Go remains ~5–10 ms ahead.

2 · Response-time consistency & tail latency



Consistency is Go's clearest win. Lower coefficient of variation and lower worst-case latency mean fewer surprises under real traffic. .NET's spikes on the lightweight Server-Time call (max 412 ms vs a 160 ms median) and on Symbol-List (max 505 ms) are the kind of jitter that shows up as occasional slow ticks.

3 · Throughput & concurrency scaling



Concurrency	Impl	Throughput (req/s)	Mean latency (ms)	Errors
1	.NET	5.1	197	0%
	Go	5.5	182	0%
6	.NET	27.9	212	0%
	Go	29.8	198	0%
20	.NET	28.1	698	0%
	Go	29.9	655	0%
50	.NET	26.2	1849	0%
	Go	25.1	1921	0%

Throughput is flat from concurrency 6 upward for both servers, and rising concurrency only inflates latency (work queues behind the browser's 6 connections). That plateau is a property of the test client, not the servers: Go's own metrics showed only 21 goroutines and ~0 meaningful CPU movement throughout, i.e. it was coasting. Conclusion: neither server was stressed, and a definitive throughput/scalability verdict requires a co-located native load test. On the offered load both are equivalent (~28–30 req/s, 0 errors).

4 · Reliability & error rate

Zero failed requests across all latency and load phases on both servers (thousands of requests each). One transient 10% error burst observed on Go at concurrency 20 coincided exactly with a local browser-extension disconnect and disappeared on a clean re-run — attributed to the client, not the server.

5 · WebSocket behaviour & stability

Metric	.NET (:5063)	Go (:5070)
Connect / handshake time	~160 ms	~163 ms
Time to first message	~199 ms*	~214 ms
Push cadence (GetQuotes)	~3.0 s (server-driven)	~3.0 s (server-driven)
20 s soak — early disconnects	0	0
10 concurrent sockets — opened	10 / 10	10 / 10
Concurrent — drops / errors	0 / 0	0 / 0

*A one-off 489 ms first-message was seen on .NET's very first WebSocket (cold start / JIT warm-up); it disappeared on every subsequent connection. The streams are functionally identical: both push a quote roughly every 3 seconds and held all

connections open without a single drop.

6 · Resource usage & observability

Metric	Go (:5070) measured	.NET (:5063)
Resident memory (RSS)	67 MB (stable under load)	not exposed
Live heap in use	~4 MB	not exposed
Goroutines / threads	17 idle → 21 under load / 9–10	n/a
Runtime	go1.26.3	ASP.NET (version not exposed)
/healthz /readyz	200 / 200	404 / 404
/metrics (Prometheus)	200	404
Status page (/)	200	not served

Go's footprint is small and, importantly, fully observable: liveness/readiness probes for orchestration (Kubernetes etc.) and Prometheus scraping are built in. Goroutine count returned to baseline after testing — no connection or memory leak. The .NET service offers none of these endpoints, so in production it is harder to health-check, autoscale or monitor. This operability gap is the most concrete differentiator found.

7 · Functional equivalence

Responses match in substance — e.g. Last Quote returned identical bid 1.14617 / ask 1.14618 on both. Minor, non-breaking schema differences worth noting for clients that parse strictly: Market Depth uses `datetime` (.NET) vs `datetime_msc` (Go); Server Time uses `unixTimestamp` (.NET) vs `UnixTimestamp` (Go); and Go appends a trailing newline to JSON bodies. Behaviour is otherwise interchangeable.

8 · Scorecard & recommendation

Criterion	Winner	Margin
Mean latency	Go	slight (~3–6%)
Latency consistency / tail	Go	clear
Throughput (this test)	tie	client-capped
Scalability under load	inconclusive	needs native test
Error rate / reliability	tie	both 0%
WebSocket latency & stability	tie	equivalent
Resource footprint	Go	measurable & small
Observability / operability	Go	decisive (.NET none)

Bottom line. If the decision rested only on raw speed, this would be a coin-flip — the two gateways are within single-digit-percent of each other on latency and tied on throughput and WebSockets. The deciding factors are everything around the request: Go delivers steadier latency with far smaller tails, runs in ~67 MB with a handful of goroutines, and ships the health, readiness and metrics endpoints needed to operate and scale a service safely. The .NET build matches it functionally but is jitterier and effectively a black box in production. Recommend standardising on the **Go** gateway, and — before committing to any capacity numbers — re-running the throughput and scalability tests with a native load generator co-located with the host to remove the browser ceiling.

Environment: client = Chrome on macOS (user machine); host 46.62.247.67, .NET :5063 / Go :5070; symbol EURUSD, source=tv; bearer auth. Latency n=100/endpoint after warm-up; throughput = async worker pool; Go metrics from /metrics. Figures reflect a browser client over the public internet and include network + CORS overhead; treat absolute milliseconds as

relative, not co-located, measurements.